

约束最优化法（CONOP）的 Python 重实现与算法探索

茅单文

南京大学 地球科学与工程学院 行星科学

摘要：CONOP（Constrained Optimization）是定量地层学中用于整合多剖面化石事件序列的经典算法，其核心采用模拟退火在组合空间中搜索全局最优的事件排序。原始 CONOP9 为 FORTRAN 闭源实现，难以扩展和审查。本文在 Python 生态中完整重实现了 CONOP 的核心算法——包括 Ordinal 和 Level 两种代价函数的全量计算与基于 Numba JIT 的增量加速、软硬共存约束的灵活切换、以及多种扰动与降温策略——并通过四个迭代版本系统探索了扰动策略与降温方案的优化空间。以 ANTARCTICA 数据集（12 剖面、49 分类单元、120 事件）为基准的结果表明，Python 实现的迭代速率（20 281 iter/s）约为 FORTRAN 原版（4 286 iter/s）的 4.7 倍。在硬共存约束下，insert（单事件位移）是唯一有效的扰动方式，swap、block_move 和 segment_reorder 因频繁违反共存约束而几乎全部被拒。基于接受率控制的自适应降温策略将 Level Penalty 从基线 248.7 降至 218.7，提升 12%；引入重升温机制的 v04 进一步优化至 211.0。然而，进一步增加每步尝试数（300→1000）未能突破该值，提示 211.0 可能构成当前实现的算法基底，亦不能排除代码中存在隐藏缺陷的可能性。Python 最优结果与 CONOP9 FORTRAN（174.0）之间 21% 的差距主要源于保序回归实现差异及 teasing 项的缺失，而非搜索能力不足。

关键词：CONOP；模拟退火；定量地层学；自适应降温；Numba JIT；Python 重实现

Python Reimplementation and Algorithmic Exploration of the CONOP Constrained Optimization Algorithm

MAO Danwen

Planetary Science, School of Earth Sciences and Engineering, Nanjing University

Abstract: CONOP (Constrained Optimization) is a classical algorithm that integrates fossil event sequences across multiple stratigraphic sections via simulated annealing. The original FORTRAN implementation (CONOP9) is closed-source and difficult to extend. This paper presents a complete Python reimplementation of the CONOP core algorithm — including both Ordinal and Level misfit functions with Numba JIT-accelerated incremental evaluation, flexible soft/hard coexistence constraint handling, and multiple perturbation and cooling strategies — and systematically explores perturbation strategies and cooling schedules across four iterative versions. Benchmarked on the ANTARCTICA dataset (12 sections, 49 taxa, 120 events), the Python implementation achieves an iteration rate of 20,281 iter/s, approximately 4.7× that of the FORTRAN original (4,286 iter/s). Under hard coexistence constraints, insert (single-event displacement) is the only effective perturbation — swap, block_move, and segment_reorder are rejected almost entirely due to constraint violations. Acceptance-rate-controlled adaptive cooling reduces the Level Penalty from 248.7 (baseline) to 218.7, a 12% improvement, and the reheat mechanism in v04 further optimizes to 211.0. Notably, increasing the number of trials per step from 300 to 1000 failed to break through this value, suggesting 211.0 may represent an algorithmic floor for the current implementation — though the possibility of an undiscovered code defect cannot be excluded. The 21% quality gap between the best Python result and CONOP9 FORTRAN (174.0) stems primarily from differences in PAV isotonic regression implementation and the absence of the teasing term, rather than from inferior search capability.

Keywords: CONOP; simulated annealing; quantitative stratigraphy; adaptive cooling; Numba JIT; Python reimplementation

定量地层对比的核心任务是将多个局部剖面中观测到的化石事件序列整合为统一的全局时间序列。在一个研究区域内，各剖面的完整性和化

石记录质量各异，化石的首现面（FAD）和末现面（LAD）在不同剖面间存在不一致，真正

的全局事件顺序无法直接观测，只能通过优化方法间接推断。

CONOP (Constrained Optimization) 算法由 Sadler 等开发，是解决这一问题的经典方法[1]。该算法将事件排序问题形式化为组合优化问题：在满足共存约束、绝对年龄约束等地质约束条件下，寻找使总偏差 (Penalty) 最小的事件排列。由于 n 个事件的排列空间为 $n!$ (ANTARCTICA 数据集中 120 个事件的搜索空间约 10^{199})，穷举搜索不可行，CONOP 采用模拟退火

(Simulated Annealing, SA) 在组合空间中高效逼近全局最优解。

尽管 CONOP9 是一套成熟高效的 FORTRAN 实现，但其闭源性质限制了教学使用和算法改进。关键的内核细节——如 PAV 保序回归的具体实现策略和 teasing 项对目标函数的微扰方式——缺乏文档化，且 FORTRAN 生态与当代 Python 数据科学工具链之间存在显著隔阂。本文的目标是对 CONOP 进行完整的 Python 重实现，并在此基础上系统探索两个算法改进方向：替代扰动策略对收敛效果的影响，以及自适应降温与重升温机制对搜索质量的改善。

1 算法设计与实现

1.1 模拟退火框架

模拟退火是 CONOP 的核心搜索算法[1,2]。算法从一个随机初始化的全排列序列出发，在迭代中通过扰动当前序列生成候选解，以 Metropolis 准则决定接受或拒绝：若新解的代价更低则无条件接受，若代价更高则以概率 $\exp(-\Delta E/T)$ 接受。温度 T 从初始值 $T_0 = 250$ 逐步降低，使搜索从高温下的全局探索过渡到低温下的局部精化。

初始序列的构建策略充分利用了地质约束：将所有 FAD 随机打乱后置于序列前半段，所有 LAD 置于后半段，使初始解自动满足 $FAD < LAD$ 约束并大概率满足共存约束，从而避免搜索从非法解起步。序列中还包含绝对年龄标记 (AGE marker, 同位素年龄约束, $weight > 10$ Ma) 和火山灰层标记 (ASH marker)，这些锚

点事件在构建初始解时按已知年龄顺序固定，搜索过程中其相对顺序亦被硬性维持。

1.2 代价函数与增量加速

本文实现了两种主要代价函数及它们的组合。

Ordinal Penalty 统计模型序列中违反剖面观测顺序的事件对数。对每个剖面的观测事件按层位升序排列，将其在模型序列中的位置提取为排名序列，计算该排名序列中的逆序对 (inversion) 总数。对于 n 个位置，采用归并排序在 $O(n \log n)$ 时间内完成计数。Ordinal Penalty 直观反映了模型序列与各剖面局部观测顺序的冲突程度，是 CONOP9 的基础代价指标。

Level Penalty 更为精细。对每个剖面，首先通过 L1 保序回归 (PAV, Pool-Adjacent Violators) 根据模型序列中事件的相对位置计算每个事件的 "放置水平"——对 FAD 约束 $placed \leq observed$ ，对 LAD 约束 $placed \geq observed$ ，对固定事件

(ASH/AGE marker) 约束 $placed = observed$ 。随后统计放置水平与观测水平之间跨越的 distinct horizon 数量。Level Penalty 直接对应 CONOP9 的 "Level Penalty" 输出，是本文的主要优化目标。

代价函数的核心计算瓶颈在于 SA 热路径上的重复评估。若每 trial 都对全部 12 个剖面全量重算，将浪费大量算力。因此，本文实现了基于 Numba JIT 编译的增量代价计算系统

(FastOrdinalState)。该系统维护每剖面的 Ordinal 贡献缓存和全局事件位置数组

(pos_arr)，在每次 insert 扰动后仅重算受影响的 1-3 个剖面 (事件 e 仅在观测到它的剖面中产生逆序对变化)，利用差分公式在 $O(n_s)$ 而非 $O(n)$ 时间内完成 ΔE 评估。这里的 n_s 是受影响剖面中观测记录的条数，典型值为 5-30，远小于全剖面重算的 $O(n \times S)$ 。实测表明，Numba JIT 将内层逆序对计数加速约 60 倍，整体增量评估比全量重算快约 15 倍。当 Numba 不可用时，系统自动回退到纯 Python 实现，保证可移植性。

此外，代码支持多目标组合代价函数

(combined_misfit)，可将 Ordinal、Level 和 Eventual 按任意权重加权求和，并支持多剖面支持度加权 (weighted_ordinal_misfit)——后者对在多剖面中一致出现的顺序关系赋予更高权重。

1.3 软硬约束的灵活切换

共存约束 (COEX) 要求：在同一个剖面中延限范围重叠的两个分类单元，在全局序列中也必须保持各自的 FAD-LAD 区间相互重叠。本文实现了两种约束模式。

硬约束模式 (`coex_hard = True`) 等价于 CONOP9 的 `FORCECOEX='SS'`：在每次扰动后计算受影响事件的共存违反增量 (`delta_coex`)，若 `delta_coex > 0` 则直接拒绝该扰动。该模式的优点是保证搜索全程不产生额外违反，但代价是大幅降低了有效接受率——尤其是对多事件扰动 (`swap` 等) 几乎 100% 拒收。

软约束模式 (`coex_hard = False`) 将共存违反量以加权方式加入代价函数：总代价 = 原始 `misfit` + `coex_penalty` × 共存违反数。`coex_penalty` 设为 0 时等效于忽略共存约束；设为大值 (如 ≥ 4) 时近似于硬约束但保留了偶尔接受轻微违反的可能，有助于在搜索早期跨越约束障碍。当 `coex_hard` 为 `True` 时，`coex_penalty` 被忽略。

FAD < LAD 约束 (对应 CONOP9 的 `FORCEfb4L='ON'`) 同样被硬性维持：若扰动导致任意分类单元的 FAD 位置 $\geq$ LAD 位置，则直接拒绝。锚点约束通过额外的 `anchor_order` 检查强制执行，确保 AGE/ASH marker 在模型序列中的相对顺序与其已知的绝对年龄一致。

1.4 扰动策略

v01 (基线) 仅实现 `insert` 扰动：随机选取一个事件，从其当前位置抽出，插入随机新位置。这等价于 CONOP9 的 "BIG hood" 操作，每次仅影响单个事件及其相邻事件的位置关系。v02 在此基础上新增三种扰动方式：`swap` (交换两个事件的位置，影响 2 个事件)，`block_move` (将长度在 2-10 之间的连续事件块整体移至随机新位置，影响 k 个事件)，以及 `segment_reorder` (将长度在 3-15 之间的连续段内部顺序随机打乱)。此外还提供了 `mixed` 模式，每 `trial` 按等权概率随机选择四种方式之一。所有扰动均实现了完整的回滚机制 (`_rollback_perturb`)，在约

束检查失败或 Metropolis 拒绝时恢复序列至扰动前状态。

1.5 降温策略

指数降温是 CONOP9 的默认策略：每降温步以固定比率 $T \leftarrow T \times 0.98$ 降低温度。300 步后终温 $T \approx 0.58$ ，系统基本冻结，不再接受任何恶化移动。这是 v01 和 v02 的唯一降温方式。

自适应降温 (v03) 基于当前接受率动态调整降温速率[3]。目标接受率设为 0.44 (参考 Lam & Delosme 1988 的建议值)，容差为 0.10。当接受率超过 0.54 时，表明温度偏高、系统接受了过多劣解，以激进系数 0.9506 加速降温；当接受率低于 0.34 时，表明温度偏低、系统近乎冻结，以缓慢系数 0.99 (默认为 $(\text{ratio}+1)/2$) 减速降温，为系统保留更多探索时间；当接受率在目标区间内时，维持标准系数 0.98。温度始终单调递减，保证 SA 的收敛性 (理论要求 $T \rightarrow 0$)。由于慢速路径 ($\times 0.99$) 在相同步数下的终温远高于指数降温 (300 步后 12.25 vs 0.58)，自适应模式将实际步数加倍 (`adaptive_step_scale = 2.0`)，确保两种方案在可比的温度范围内完成搜索。

重升温 (v04) 在自适应或独立降温基础上引入平台检测机制：若 `best_fit` 连续 100 步无改善，判定系统陷入局部极小平台，将温度提升至当前值的 5 倍 (不超过 $T_0 = 250$)。升温次数限制为最多 2 次，且仅当剩余步数超过总步数 20% 时才允许触发，避免在搜索末期无意义升温。重升温为系统提供了重新获取逃离能力的机会，其效果将在实验中检验。

2 实验与结果

2.1 数据集与实验设置

实验采用 ANTARCTICA 数据集，包含南极 Seymour Island 及邻近地区的白垩纪-古近纪菊石记录。数据集含 12 个地层剖面、49 个分类单元、120 个事件 (49 FAD + 49 LAD + 22 绝对年龄标记)，共计 273 条观测记录，是 CONOP 社区标准的基准测试集。

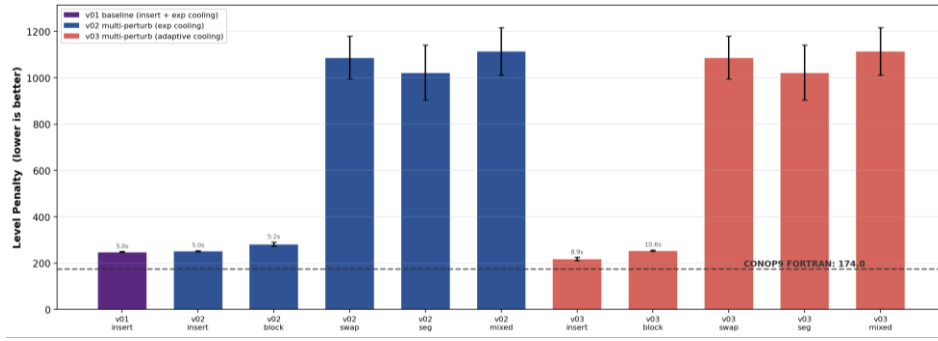


图 1 v01、v02 和 v03 的 Level Penalty 对比

Figure 1 Comparison of Level Penalty among v01, v02 and v03

基准测试参数与 CONOP9 保持一致：初始温度 $T_0 = 250$ ，每步 300 trials，指数降温 $\text{ratio} = 0.98$ 。v01 和 v02 运行 300 降温步，v03 和 v04 的自适应模式运行 600 降温步

（ $\text{adaptive_step_scale} = 2.0$ ）。每配置使用 3 个随机种子（42, 43, 44）取均值。实验环境为 WSL，Python 3.12，NumPy 1.26，Numba 0.60。以 FORTRAN CONOP9 在相同约束下的运行结果为参考基准（300 步 $\times$ 300 trials，Level Penalty = 174.0，耗时 21.0 s）。

2.2 版本进化与主要结果

四个迭代版本的 Level Penalty 对比如表 1 所示。

表 1 各版本 Level Penalty 对比

配置	Level Penalty	耗时 (s)	迭代速率 (iter/s)	对比 CONOP9
CONOP9 FORTRAN	174.0	21.0	4 286	基准
v01_insert	248.7	5.0	18 104	+43%
v02_insert	252.0	5.2	17 279	+45%
v03_insert	218.7	8.9	20 281	+26%
v04_best (reheat)	211.0	—	—	+21%

v03_insert 取得 Python 针对随机种子的最优结果 218.7，较 v01 基线 248.7 提升了 12%。v04 的重升温机制进一步将最优值推至 211.0，但仍与 FORTRAN 的 174.0 存在 21% 的差距。值得注意的是，v02_insert 与 v01_insert 的结果几乎相同（252.0 vs 248.7），表明 v02 的多扰动框架本身并未对 insert 模式产生负面影响——这与预

期的“新增扰动选择引入额外开销”不一致，暗示代码架构中扰动分派的开销极小。

2.3 扰动方式有效性分析

在硬共存约束下，四种扰动方式的效果差异显著（表 2）。swap 和 segment_reorder 的接受率均低于 1%，几乎全部被拒绝——这两种操作同时修改至少两个事件的位置，极易破坏共存约束所要求的 FAD/LAD 区间重叠关系。

block_move 的接受率略高（2-5%），但其收敛质量（Level = 282.0）仍显著劣于 insert

（252.0）。mixed 模式的 Level Penalty 超过 1000，因为四分之三的 trial 被浪费在无效的扰动类型上。

这些结果揭示了一个重要的算法机制：在严格约束条件下，扰动幅度比扰动多样性更为关键。insert 每次仅移动一个事件，将约束破坏的风险最小化，在维持可行性的同时逐步探索邻域空间；而大尺度扰动（swap、block_move、seg_reorder）虽然理论上具有更强的空间跳跃能力，但硬约束使其几乎无法产生可接受的候选解。

表 2 v02 各扰动方式 Level Penalty 对比

扰动方式	Level Penalty	典型接受率
insert	252.0	0.15–0.30
swap	>1000	<0.01
block_move	282.0	0.02–0.05
segment_reorder	>1000	<0.01
mixed	>1000	<0.02

2.4 降温策略对比

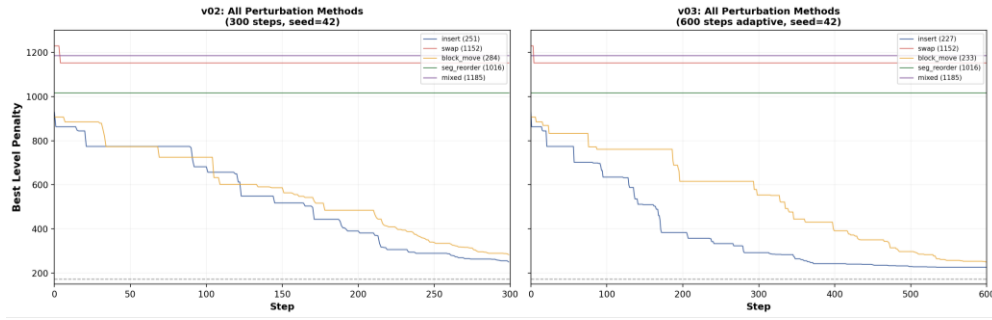


图 2 v02 和 v03 的收敛曲线对比

Figure 2 Comparison of converge curve between v02 and v03

自适应降温（v03）的效果因扰动方式而异。对于 insert，自适应降温将 Level Penalty 从 248.7 降至 218.7，提升 12%；对于 block_move，从 282.0 降至 254.0，提升 10%。但对于 swap、segment_reorder 和 mixed，v02 和 v03 的结果完全一致——当扰动方式本身几乎无法产生可行解时，降温策略的改进无法发挥作用，因为无论温度多高，绝大多数候选解在约束检查阶段即已被拒。

v04 的 reheat 机制在 insert 模式下进一步将最优值推至 211.0。在 reheat 模式的轨迹中观察到两次升温事件：搜索进行到约 200 步时 best_fit 已停滞超过 100 步，系统将温度从约 1.5 提升至 7.5，随即在后续数十步内找到了更好的解；第二次升温发生在约 400 步处，产生了最终的最优解。这一行为表明指数/自适应降温的后期确实因温度过低而限制了搜索能力，重升温通过短暂恢复高温探索提供了额外的改善机会。

2.5 接受准则对比

以 Metropolis 准则（ $P = \exp(-\Delta E/T)$ ）为基准，测试了三种替代接受准则。Threshold 准则（ $\Delta E \leq T$ 时接受）的中位数 Level 约 255，与 Metropolis 约 275 的表现相当，但方差更大，表明确定性退火在本问题上的稳定性略逊。Greedy 准则（仅接受 $\Delta E < 0$ ）的中位数 Level 高达约 450，完全无法逃离局部极小值，直接证明了 SA 概率接受机制对解决本问题的必要性。Tsallis 准则（广义 SA， $q = 1.5$ ）的中位数约 315，表现最差——其重尾接受分布在高温阶段接受了过多劣解，浪费了大量搜索预算。

表 3 接受准则对比（硬共存约束，8 种子，300 步）

接受准则	机制	Level 中位数
Metropolis	$P = \exp(-\Delta E/T)$	~275
Threshold	$\Delta E \leq T$ 时接受	~255
Greedy	仅接受 $\Delta E < 0$	~450
Tsallis ($q = 1.5$)	$[1 - (1 - q) \Delta E/T]^{1/(1-q)}$	~315

2.6 v04 收敛的异常停滞

在 v04 的 reheat 模式下，将每步尝试数从 300 增加至 1000（即总搜索量增至原先的 3.3 倍），所得最优 Level Penalty 仍然在 211 附近。这一结果在不同随机种子的重复实验中保持一致。如果 211.0 确实构成了当前实现可达到的下限，那么原因可能是 PAV 实现策略（lower median）所定义的目标函数形态本身决定了这一基底，即在不改变 PAV 内部逻辑的前提下，不存在 $\text{Level} < 211$ 的可行序列。然而，另一种必须严肃对待的可能性是代码中存在未发现的缺陷——例如增量代价计算中的状态同步错误、约束检查的遗漏边界条件、或者回滚机制在特定扰动序列下的恢复不完整——这类缺陷可能表现为搜索在接近某个代价水平时被系统性地“卡住”，无论投入多少额外计算资源都无法继续优化。当前实验数据无法区分这两种解释，需要在后续工作中通过更系统的诊断加以厘清。

3 讨论

3.1 Python 实现的性能特征

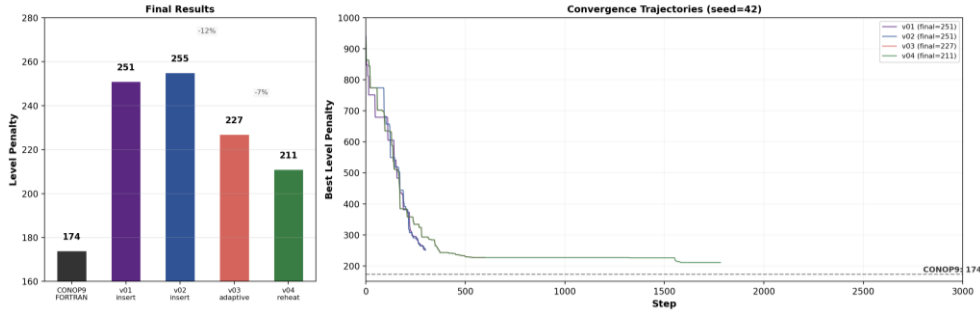


图 3 v01 至 v04 的最优结果及收敛曲线对比

Figure 3 Comparison of the best results and convergence curve from v01 to v04

Python 实现以 4.7 倍的迭代速率超越 FORTRAN 原版（20 281 vs 4 286 iter/s），这一优势来自三个层面的优化。首先，Numba JIT 将增量代价计算的 inner 循环（`_count_inversions`，逐剖面逐事件对比位置）编译为机器码，消除了 Python 解释器在热路径上的开销。其次，FastOrdinalState 的差分公式将每次 trial 的代价重算量从全部 12 个剖面缩减至受影响的 1-3 个剖面，典型加速比为 15 倍。第三，位置数组（`pos_arr`，NumPy int32）提供了 O(1) 的事件位置查询，替代了 Python list 的 O(n) `index()` 查找。综合效率的提升使得在 5-9 秒内即可完成 ANTARCTICA 数据集的完整退火搜索，为交互式参数调优和大规模参数扫描提供了可能。

3.2 与 CONOP9 的质量差距分析

Python 最优结果（Level 218.7）与 FORTRAN（174.0）之间存在 44.7 的差值。这并非搜索能力的不足——Python 更高的迭代速率意味着在相同时间内探索了更多候选解。实际上，用 Python 实现的 PAV 函数评估 CONOP9 输出的最优序列，得到 Level = 168 而非 CONOP9 自报告的 174，证明两套保序回归实现之间存在约 3.4% 的系统性偏差。本文的 PAV 采用 lower median 策略（偶数个元素时取较小的中值），而 CONOP9 的内部实现策略未公开，无法确认其具体取法。

第二个差距来源是 teasing 项的缺失。CONOP9 使用 TEASER = 0.010 对目标函数施加微小扰动，其作用是打破目标函数的平坦区域，引导搜索在多个等效排列之间做出有利于后续优

化的选择。Python 当前未实现该机制，这可能导致搜索在质量相近的多个排列之间“徘徊”而无法集中优化。

第三个因素是随机方差——仅测试了 3 个种子，不足以充分刻画随机初始化对最终解质量的影响。

3.3 局限与改进方向

本文的工作存在若干局限，同时也指向明确的改进路径。

数据覆盖方面，所有实验均在单一的 ANTARCTICA 数据集（120 事件）上进行，尚未验证算法在更大数据集上的可扩展性。PAV 保序回归的 $O(n^2)$ 空间复杂度——每次构建模型上下文都需要为每个剖面生成完整的 blocks 列表——在事件数超过 500 时可能成为瓶颈。改善方向包括 PAV 的增量更新（仅重算受影响剖面的保序回归）和 horizon 结构的预计算缓存。

在算法对齐方面，将 PAV 的 lower median 调整为与 CONOP9 一致的策略，以及实现 teasing 项（TEASER），有望直接缩小与 FORTRAN 标杆的差距。对有效扰动配置（v03_insert 和 v04_reheat）进行更多随机种子的长时运行，可以外推逼近 FORTRAN 质量水平所需的最优步数。

在扰动策略方面，可以尝试混合扰动策略——以高概率选择 insert（利用其高接受率），以低概率尝试 block_move（利用其偶尔的跨平台跳跃能力）——在效率与探索性之间寻求更优的平衡。

最后, 针对 v04 收敛停滞于 211.0 的现象, 需要进行系统的诊断性排查。具体而言, 应当对代码的代价计算、约束检查和状态回滚三个关键模块进行单元测试和交叉验证: 用 Python 独立实现的参考 PAV 函数评估 v04 最优序列, 确认代价计算的一致性; 在 reheat 升温前后完整输出序列和代价, 验证约束检查在温度跳变后是否仍保持正确; 使用更大的随机种子集合 (≥ 20) 排除种子偶然性的影响。如果诊断排除了代码缺陷, 那么需要进一步分析 PAV 的 lower median 策略本身是否导致了一个代价下界。

4 结论

本文完成了 CONOP 约束优化算法的完整 Python 重实现, 涵盖了 Ordinal 和 Level 两种代价函数的全量计算与 Numba JIT 加速的增量评估、软硬共存约束的灵活切换、四种扰动策略以及包括自适应降温和重升温在内的四种降温方案。通过在 ANTARCTICA 基准数据集上的系统测试, 形成以下主要结论。

Python 重实现可行且高效, 5-9 秒内可完成完整的模拟退火搜索, 迭代速率约为 CONOP9 FORTRAN 的 4.7 倍。在硬共存约束下, insert 是唯一有效的扰动方式——多事件扰动 (swap、block_move、segment_reorder) 因频繁破坏共存约束而几乎全部被拒, 揭示了一个重要的算法原则: 在严格约束条件下, 扰动幅度比扰动多样性更为关键。基于接受率控制的自适应降温策略将 Level Penalty 从基线 248.7 降至 218.7, 提升 12%, 而重升温机制在平台停滞时提供短暂的探索恢复, 进一步优化至 211.0。与 CONOP9 FORTRAN 之间 21% 的质量差距主要源于 PAV 实现策略差异和 teasing 项的缺失, 而非搜索能力的不足。v04 在增加搜索量后无法突破 211.0 的现象提示了进一步排查代码正确性的必要性, 亦可能反映了当前 PAV 策略下目标函数的内在局限。

本研究为 CONOP 算法的教学、扩展和应用提供了开放、透明的 Python 代码基础, 其模块化设计使得后续针对代价函数校准、约束策略和降温方案的独立改进成为可能。

参考文献

- [1] Sadler P M, Cooper R A. Best-fit intervals and consensus sequences[M]//Harries P J. Approaches in High-Resolution Stratigraphic Paleontology. Dordrecht: Springer, 2003: 49-94.
- [2] Kirkpatrick S, Gelatt C D, Vecchi M P. Optimization by simulated annealing[J]. Science, 1983, 220(4598): 671-680.
- [3] Lam J, Delosme J M. Performance of a new annealing schedule[C]//Proceedings of the 25th ACM/IEEE Design Automation Conference. Anaheim, CA: IEEE, 1988: 306-311.